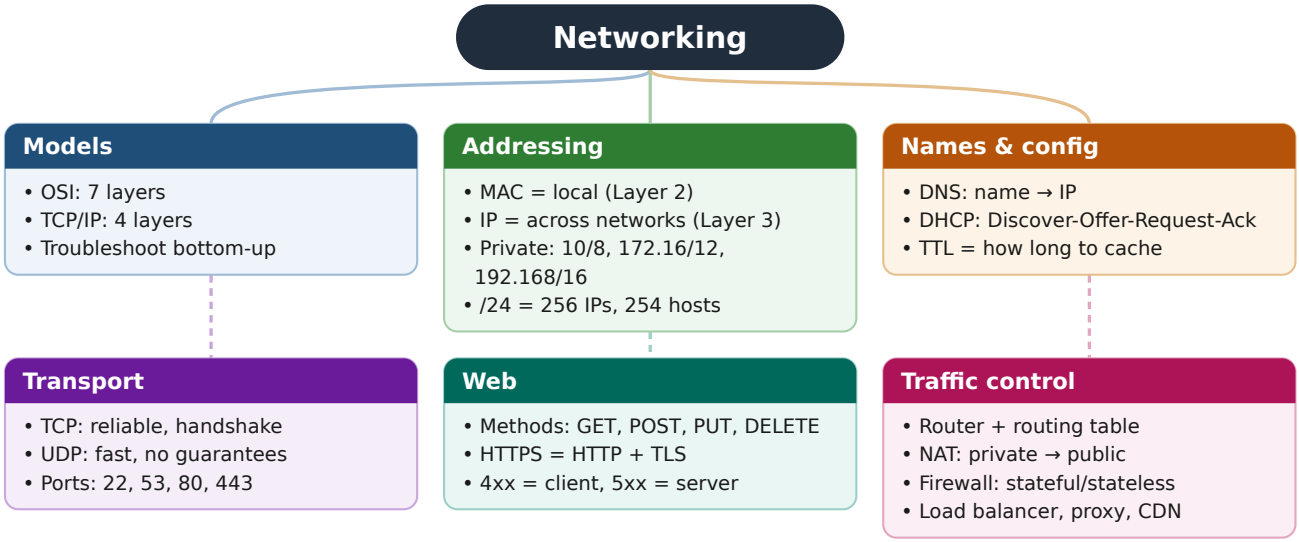
2. Networking Basics for DevOps

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 2 of 18 · Thirupathi Keloth · v2.0

Contents

2.0 Why networking matters for DevOps
2.1 Network fundamentals
2.2 OSI and TCP/IP models
2.3 Ethernet, MAC and ARP
2.4 IPv4 addressing
2.5 Binary and subnetting (complete method)
2.6 IPv6 essentials
2.7 Routing, default gateway and NAT
2.8 TCP and UDP in depth
2.9 DNS in depth
2.10 DHCP
2.11 HTTP in depth
2.12 TLS and HTTPS in depth
2.13 SSH (protocol view)
2.14 Firewalls and network security
2.15 Load balancing
2.16 Proxies, reverse proxies, API gateways and CDNs
2.17 VPNs and private connectivity
2.18 Networking in cloud, containers and Kubernetes
2.19 Networking tools reference
2.20 Network troubleshooting
2.21 Interview questions and answers
2.22 Self-check: I can...



2.0 Why networking matters for DevOps

Most production incidents involve the network somewhere: DNS not resolving, a security group blocking a port, a certificate expiring, a load balancer marking targets unhealthy, a pod that can't reach a database. Cloud (VPCs), containers (bridges, port publishing) and Kubernetes (Services, Ingress, NetworkPolicies) are all networking. These notes cover what a DevOps/Cloud/SRE engineer uses daily. The separate **Computer Networking** book goes deeper into switching, routing protocols and CCNA topics.

2.1 Network fundamentals

Term	Meaning
Network	Devices connected to exchange data
Host / node	Any device with an address (server, laptop, container, router)
LAN / WAN	Local network (one site) / wide area network (connects sites; the internet)
Packet	A unit of data with headers (addresses) and payload
Protocol	Agreed rules for communication (IP, TCP, HTTP, DNS)
Bandwidth / throughput	Maximum capacity / actually achieved rate
Latency / RTT	One-way delay / round-trip time (ms)
Jitter / packet loss	Variation in delay / packets that never arrive

Device	Layer	Job
Switch	2	Connects devices in a LAN using MAC addresses
Router	3	Connects networks; forwards packets by IP address
Firewall	3–7	Allows/blocks traffic by rules
Load balancer	4–7	Spreads traffic across servers
Access point	1–2	Wi-Fi
Proxy / reverse proxy	7	Forwards requests on behalf of clients / servers

2.2 OSI and TCP/IP models

OSI layer	TCP/IP layer	Data unit	Addressing / examples	DevOps view: what breaks here
7 Application	Application	Data	HTTP, DNS, SSH, TLS-protected apps	Wrong URL, 4xx/5xx, bad API response
6 Presentation	Application	Data	TLS, encoding, compression	Certificate errors, encoding issues
5 Session	Application	Data	Sessions	Session timeouts
4 Transport	Transport	Segment / datagram	**Ports**, TCP, UDP	Port closed, firewall, connection refused/timeouts
3 Network	Internet	Packet	**IP addresses**, routing, ICMP	Wrong IP/route, no gateway, NAT issues
2 Data link	Link	Frame	**MAC addresses**, Ethernet, ARP, VLANs	ARP/VLAN problems (rare in cloud)
1 Physical	Link	Bits	Cables, fibre, radio	Cable/Wi-Fi down

Mnemonic (7→1): **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.

Encapsulation

```
App data                                    "GET /index.html"
+ TCP header (src port 51514, dst port 443)       → segment
+ IP header  (src 10.0.1.5, dst 93.184.216.34)    → packet
+ Ethernet header (src MAC, dst MAC = gateway)    → frame → bits on the wire
```

Each router hop **removes and rebuilds** the Ethernet header (new MACs) but keeps the **IP addresses** (unless NAT changes them) and the TCP ports.

2.3 Ethernet, MAC and ARP

- **MAC address:** 48-bit hardware address (`02:5e:3a:1b:4c:7d`), used only inside the local network.
- **Switch logic:** learn source MACs per port, forward to the known port, flood unknown/broadcast.
- **ARP:** finds the MAC for an IPv4 address in the local network ("Who has 10.0.1.1? Tell 10.0.1.5"). For destinations outside the subnet, the host ARPs for its **default gateway**.
- **VLANs** split one physical switch into separate logical networks (one VLAN = one subnet). In the cloud, you work with subnets and security groups instead of VLANs.

Commands: `ip neigh` (Linux), `arp -a` (Windows).

2.4 IPv4 addressing

- 32 bits written as four decimal octets: `192.168.10.25` .
- Split into **network** and **host** parts by the subnet mask / prefix length (`/24`).

Range	Purpose
`10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16`	**Private** (RFC 1918): not routed on the internet; used in LANs, VPCs, Docker
`127.0.0.0/8` (`127.0.0.1`)	Loopback: "this host" (`localhost`)
`169.254.0.0/16`	Link-local / APIPA (no DHCP answer); in AWS also the **instance metadata** service `169.254.169.254`
`100.64.0.0/10`	Carrier-grade NAT; sometimes used in Kubernetes/cloud for pod networks
`0.0.0.0`	"Any address" (a server listening on `0.0.0.0` accepts on all interfaces); `0.0.0.0/0` = default route
`255.255.255.255`	Limited broadcast
Everything else (mostly)	**Public** addresses routable on the internet

Public vs private in practice: your laptop has a private IP behind your home router; the router has a public IP from the ISP and uses **NAT** so many private devices share it. Cloud instances have private IPs in the VPC and optionally a public IP mapped by the cloud provider.

Listening addresses matter: a service bound to `127.0.0.1:8000` is reachable only from the same machine; bound to `0.0.0.0:8000` it is reachable from the network (if firewalls allow). This is a very common cause of "works on the server, not from outside" and "container port not reachable".

2.5 Binary and subnetting (complete method)

2.5.1 Number systems

System	Base	Digits	Used for
Decimal	10	0–9	How people write IPv4: `192.168.1.10`
Binary	2	0, 1	How computers actually store IPs and masks
Hexadecimal (hex)	16	0–9, A–F	MAC addresses, IPv6

Unit	Size
Bit	One 0 or 1
Nibble	4 bits (= one hex digit)
Byte / **octet**	8 bits. An IPv4 address = 4 octets = 32 bits

2.5.2 Place values (the key to everything)

Each position in an 8-bit octet has a value. **Memorize this row:**

Position	8th	7th	6th	5th	4th	3rd	2nd	1st
Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**

All 8 bits on: 128+64+32+16+8+4+2+1 = **255** → that's why an octet goes from 0 to 255.

Powers of 2 (for counting addresses):

2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{11}	2^{12}	2^{16}
2	4	8	16	32	64	128	256	512	1024	2048	4096	65536

2.5.3 Binary → decimal

Write the place values above the bits and **add the values where the bit is 1**.

Binary	128	64	32	16	8	4	2	1	Sum
11000000	1	1	0	0	0	0	0	0	128+64 = **192**
10101000	1	0	1	0	1	0	0	0	128+32+8 = **168**
00001010	0	0	0	0	1	0	1	0	8+2 = **10**
01001101	0	1	0	0	1	1	0	1	64+8+4+1 = **77**

2.5.4 Decimal → binary (subtraction method)

Go left to right. **If the number is ≥ the place value: write 1 and subtract. Otherwise write 0.**

Example: 172

Place value	128	64	32	16	8	4	2	1
Fits?	172 ≥ 128 ✓	44 < 64 ✗	44 ≥ 32 ✓	12 < 16 ✗	12 ≥ 8 ✓	4 ≥ 4 ✓	0 ✗	0 ✗
Remaining	44	44	12	12	4	0	0	0

Place value	128	64	32	16	8	4	2	1
Bit	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**0**

172 = **10101100**. Check: 128+32+8+4 = 172 ✓

Example: 200 → 128 ✓ (72 left) → 64 ✓ (8 left) → 32 ✗ → 16 ✗ → 8 ✓ (0 left) → 4 ✗ → 2 ✗ → 1 ✗ = **11001000**

Full address:

```
192      .168      .10       .77
11000000 .10101000 .00001010 .01001101
```

2.5.5 Hexadecimal

One hex digit = 4 bits. A=10, B=11, C=12, D=13, E=14, F=15.

Hex	Binary	Hex	Binary	Hex	Binary	Hex	Binary
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

- **Hex → decimal:** `1A` = 1×16 + 10 = **26**. `FF` = 15×16 + 15 = **255**. `2F` = 2×16 + 15 = **47**.
- **Hex → binary:** convert each digit separately: `0DB8` = 0000 1101 1011 1000.
- Used in MAC addresses (`00:1A:2B:...`) and IPv6 (`2001:db8::1`).

2.5.6 The subnet mask in binary

An IPv4 address has a **network part** (which network) and a **host part** (which device). The **subnet mask** marks them: **1 = network bit, 0 = host bit**. The 1s are always continuous from the left.

```
/24  = 11111111.11111111.11111111.00000000 = 255.255.255.0     (24 network bits, 8 host bits)
/27  = 11111111.11111111.11111111.11100000 = 255.255.255.224   (27 network bits, 5 host bits)
```

Prefix length (`/27`) = number of 1s. **Host bits (h)** = 32 − prefix.

Only these values can appear in a mask octet:

Mask octet	Binary	Network bits in octet	Block size (256 − value)
0	00000000	0	256
128	10000000	1	128
192	11000000	2	64
224	11100000	3	32
240	11110000	4	16
248	11111000	5	8
252	11111100	6	4
254	11111110	7	2
255	11111111	8	1

The formulas:

Want to know	Formula	Example /27 (h = 5)
Total addresses in a subnet	2^h	$2^5 = 32$
Usable hosts	$2^h - 2$ (network + broadcast reserved)	$32 - 2 = 30$
Number of subnets when borrowing s bits	2^s	/24 → /27 borrows 3 bits → $2^3 = 8$ subnets

2.5.7 How the network address is found: the AND operation

Routers find the network by doing a bitwise **AND** of the IP and the mask. **AND rule: 1 AND 1 = 1, anything else = 0.**

Example: 192.168.10.77/27 (only the last octet changes, because the first three mask octets are 255)

```
IP   last octet   77 = 0 1 0 | 0 1 1 0 1
Mask last octet  224 = 1 1 1 | 0 0 0 0 0
                       network | host bits
AND result           = 0 1 0 | 0 0 0 0 0 = 64   → Network   192.168.10.64

Set all host bits to 1 = 0 1 0 | 1 1 1 1 1 = 95   → Broadcast 192.168.10.95
```

Result	Value
Network address (all host bits 0)	192.168.10.64
First usable host (network + 1)	192.168.10.65
Last usable host (broadcast − 1)	192.168.10.94
Broadcast address (all host bits 1)	192.168.10.95
Usable hosts	$2^5 - 2 = 30$

2.5.8 The fast method: block size

Use this in exams and real work. No binary needed once you understand 6.7.

1. **Find the interesting octet:** the octet where the mask is **not** 255 or 0.

Prefix	/1–/8	/9–/16	/17–/24	/25–/32
Interesting octet	1st	2nd	3rd	4th

2. **Block size** = 256 − mask value in that octet.
3. Subnets start at **multiples of the block size**: 0, block, 2×block...
4. **Network** = the largest multiple that is ≤ the IP's value in that octet. Octets to the right become 0.
5. **Broadcast** = next network − 1 (octets to the right become 255).

Example 1: 10.20.30.200/26 - /26 → 4th octet, mask 192 → block **64** → subnets .0, .64, .128, **.192** - 200 is in the .192 block → **Network 10.20.30.192**, broadcast **10.20.30.255**, hosts .193 – .254 (62 hosts)

Example 2: 172.16.45.10/20 - /20 → 3rd octet, mask 255.255.**240**.0 → block **16** → 0, 16, **32**, 48... - 45 is in the 32 block → **Network 172.16.32.0**, broadcast **172.16.47.255** - Hosts 172.16.32.1 – 172.16.47.254 → $2^{12} - 2 =$ **4,094**

Example 3: 192.168.5.130/30 - block 4 → .128, .132 → **Network .128**, hosts **.129, .130**, broadcast **.131** (typical router-to-router link)

Example 4: 10.5.200.9/14 - /14 → 2nd octet, mask 255.**252**.0.0 → block **4** → 0, **4**, 8... - 5 is in the 4 block → **Network 10.4.0.0**, broadcast **10.7.255.255**, $2^{18} - 2 = 262{,}142$ hosts

2.5.9 Designing subnets

A) "I need N subnets" → find the smallest s where $2^s \geq N$, add s to the prefix.

Need 6 subnets from 192.168.1.0/24 → $2^3 = 8 \geq 6$ → borrow 3 bits → **/27** → block 32, 30 hosts each:

#	Network	Usable range	Broadcast
1	192.168.1.0/27	.1 – .30	.31
2	192.168.1.32/27	.33 – .62	.63
3	192.168.1.64/27	.65 – .94	.95
4	192.168.1.96/27	.97 – .126	.127
5	192.168.1.128/27	.129 – .158	.159
6	192.168.1.160/27	.161 – .190	.191
7–8	.192/27, .224/27	spare	

B) "I need H hosts per subnet" → find the smallest h where $2^h - 2 \geq H$, prefix = 32 − h.

Hosts needed (up to)	2	6	14	30	62	126	254	510	1022	2046	4094
Prefix	/30	/29	/28	/27	/26	/25	/24	/23	/22	/21	/20

Need 500 hosts → $2^9 - 2 = 510 \geq 500$ → h = 9 → **/23**.

2.5.10 Prefix reference table

Prefix	Mask	Addresses	Usable hosts
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/20	255.255.240.0	4,096	4,094
/22	255.255.252.0	1,024	1,022
/23	255.255.254.0	512	510
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2 (point-to-point links)
/31	255.255.255.254	2	2 (point-to-point, RFC 3021)

Prefix	Mask	Addresses	Usable hosts
/32	255.255.255.255	1	Single host / loopback

2.5.11 VLSM (Variable Length Subnet Masking)

Give each subnet only the size it needs. **Allocate the largest first**, so the blocks line up.

Example from `192.168.1.0/24`:

Need	Hosts	Prefix	Subnet	Range	Broadcast
Sales	100	/25	192.168.1.0	.1 – .126	.127
IT	50	/26	192.168.1.128	.129 – .190	.191
Mgmt	20	/27	192.168.1.192	.193 – .222	.223
Link R1–R2	2	/30	192.168.1.224	.225 – .226	.227

2.5.12 Route summarization (in binary)

Combine contiguous networks into one route. Write the changing octet in binary and **count the bits that are the same in all of them**.

```
10.1.0.0  → 3rd octet 0 = 000000|00
10.1.1.0  → 3rd octet 1 = 000000|01
10.1.2.0  → 3rd octet 2 = 000000|10
10.1.3.0  → 3rd octet 3 = 000000|11
Common bits: 8 + 8 + 6 = 22  →  summary 10.1.0.0/22
```

Smaller routing tables and more stability.

2.5.13 Practice questions (answers below)

1. Convert 150 to binary.
2. Convert 11110000 to decimal.
3. Convert hex 2F to decimal.
4. Network, broadcast and host range of 192.168.50.100/28?
5. How many usable hosts in a /22?
6. Smallest subnet for 50 hosts?
7. Are 10.0.0.70/26 and 10.0.0.130/26 in the same subnet?
8. How many /24 subnets fit in 172.16.0.0/16?
9. Summarize 192.168.16.0/24 to 192.168.23.0/24 into one route.
10. Last usable host of 10.10.10.0/23?

Answers:

#	Answer	How
1	10010110	128 + 16 + 4 + 2 = 150
2	240	128 + 64 + 32 + 16
3	47	2×16 + 15
4	Network .96, hosts .97 – .110, broadcast .111	/28 → block 16 → 96 ≤ 100 < 112
5	1,022	h = 10 → $2^{10} - 2$

#	Answer	How
6	/26 (62 hosts)	/27 gives only 30
7	No	/26 blocks: 64–127 and 128–191
8	256	Borrow 8 bits → 2^8
9	192.168.16.0/21	16 = 00010000, 23 = 00010111 → 5 common bits → 16 + 5 = 21
10	10.10.11.254	/23 → block 2 in 3rd octet → network 10.10.10.0, broadcast 10.10.11.255

2.5.14 Subnetting for cloud networks

- AWS reserves **5 addresses per subnet** (network, VPC router `.1`, DNS `.2`, future `.3`, broadcast): a `/24` gives **251** usable IPs, a `/28` only 11.
- Typical VPC plan: VPC `10.0.0.0/16` → public `/24`s (`10.0.1.0`, `10.0.2.0`) → private app `/24`s (`10.0.11.0`, `10.0.12.0`) → data `/24`s (`10.0.21.0`, `10.0.22.0`), one per AZ.
- Kubernetes on AWS (VPC CNI) gives every pod a VPC IP: size node subnets generously (`/20` or larger) or use secondary CIDRs.
- **Never overlap** CIDRs between VPCs, offices and VPNs you might connect; Docker's default `172.17.0.0/16` can also clash with corporate networks.

2.6 IPv6 essentials

- 128 bits, 8 groups of hex: `2001:0db8:0000:0000:0000:ff00:0042:8329` → shortened `2001:db8::ff00:42:8329` (drop leading zeros; one `::` replaces consecutive zero groups).
- LANs use **/64** subnets. No broadcast (multicast instead), no NAT needed (huge address space), so **firewalls/security groups** are essential.

Type	Prefix	Use
Global unicast	`2000::/3`	Public addresses
Unique local	`fd00::/8`	Private
Link-local	`fe80::/10`	Local link only; every interface has one
Loopback	`::1`	This host
Multicast	`ff00::/8`	Group delivery (replaces broadcast)

Assignment: static, **SLAAC** (host builds its own address from the router's prefix), DHCPv6. **NDP** replaces ARP. In AWS, VPCs can be dual-stack; an **egress-only internet gateway** gives IPv6 outbound-only access.

2.7 Routing, default gateway and NAT

How a host decides where to send a packet

1. Is the destination in **my subnet** (compare with my IP + mask)? → send directly (ARP for its MAC).
2. Otherwise → send to the **default gateway** (router), which forwards using its routing table.

```
$ ip route
default via 10.0.1.1 dev ens5            ← everything not matched below goes to the gateway
10.0.1.0/24 dev ens5 proto kernel        ← my own subnet: deliver directly
172.17.0.0/16 dev docker0                ← Docker bridge network
```

Routing table rules: most specific match wins (**longest prefix**); `0.0.0.0/0` is the default route; each hop decreases the IP **TTL** by 1 (packet dropped at 0 → prevents endless loops; `traceroute` uses this).

Routing type	Where
Static routes	Configured manually (small networks, cloud route tables)
Dynamic routing (OSPF, BGP)	Routers learn routes from each other; BGP runs the internet and cloud hybrid links (VPN/Direct Connect)

NAT (Network Address Translation)

Type	What	Example
SNAT / PAT (masquerade)	Many private IPs share one public IP, distinguished by **source ports**	Home router; AWS **NAT Gateway**; Docker containers going out
DNAT / port forwarding	Traffic to a public IP:port is forwarded to a private IP:port	`docker run -p 8080:80`; Kubernetes NodePort; router port forward
1:1 NAT	One private ↔ one public IP	AWS public IP / Elastic IP on an instance

Consequences: inbound connections to private hosts need DNAT or a load balancer; servers see the NAT's public IP, not the client's private IP; NAT keeps connection state (tables can fill up under heavy load).

2.8 TCP and UDP in depth

Ports and sockets

- **Port:** 16-bit number (0–65535) identifying an application on a host. **Well-known** 0–1023, **registered** 1024–49151, **ephemeral/dynamic** (client side, Linux default 32768–60999).
- **Socket:** IP + port. A **connection** is identified by the 5-tuple: protocol, source IP, source port, destination IP, destination port.

Port	Service	Port	Service
20/21	FTP	443	HTTPS (TCP; HTTP/3 uses UDP 443)
22	SSH / SFTP	465/587	SMTP submission (TLS)
25	SMTP	853	DNS over TLS
53	DNS (UDP + TCP)	1433	Microsoft SQL Server
67/68	DHCP (UDP)	2049	NFS (AWS EFS)
80	HTTP	2379-2380	etcd
110/143	POP3 / IMAP	3000	Grafana (default)
123	NTP (UDP)	3306	MySQL / MariaDB
161/162	SNMP	3389	RDP
389/636	LDAP / LDAPS	5432	PostgreSQL

Port	Service	Port	Service
6379	Redis	6443	Kubernetes API server
8080	Alt HTTP (apps, Jenkins)	9090	Prometheus
9092	Kafka	10250	kubelet API
27017	MongoDB	30000-32767	Kubernetes NodePort range

TCP (Transmission Control Protocol)

Reliable, ordered, connection-oriented byte stream.

```
Client                          Server (listening on :443)
  │ ── SYN (seq=x) ──────────►  │   1. "I want to connect"
  │ ◄── SYN-ACK (seq=y, ack=x+1) │   2. "OK, I acknowledge"
  │ ── ACK (ack=y+1) ─────────► │   3. Connection ESTABLISHED
  │ ═══ data ⇄ ACKs ═══════════ │   sequence numbers, acknowledgments, retransmissions
  │ ── FIN ──────────────────►  │   graceful close (each side sends FIN + ACK)
  │ ◄── ACK, FIN ─────────────  │
  │ ── ACK ──────────────────►  │   client side waits in TIME_WAIT (~60 s on Linux)
  │ RST = abort immediately (e.g. port closed, connection reset by peer)
```

Mechanism	Purpose
Sequence numbers & ACKs	Order and confirm delivery
Retransmission	Resend lost segments (causes latency spikes on lossy networks)
Flow control (window)	Receiver tells sender how much it can accept
Congestion control	Sender slows down when the network is congested (CUBIC, BBR)

TCP states you'll see in `ss -tan`: `LISTEN` (server waiting), `SYN-SENT` (client waiting for answer: firewall dropping?), `ESTAB` (established), `TIME-WAIT` (recently closed by this side: normal; huge numbers mean many short connections → use keep-alive/pooling), `CLOSE-WAIT` (peer closed but the app hasn't: often an **application bug** leaking connections).

What errors mean:

Error	Meaning
`Connection refused`	Host reachable, but nothing listening on that port (or RST from a firewall)
`Connection timed out`	No answer at all: packets dropped (firewall/security group/NACL), wrong IP/route, host down
`Connection reset by peer`	The other side aborted the connection (crash, idle timeout, proxy)
`No route to host`	Routing problem or ICMP unreachable from a firewall

UDP (User Datagram Protocol)

Connectionless, no handshake, no delivery guarantee, low overhead. Used by DNS, DHCP, NTP, syslog, VoIP/video, gaming, QUIC/HTTP/3, VXLAN overlays. Testing UDP is harder: no "connection" to confirm (`nc -u`, application-level checks).

	TCP	UDP
Connection	Yes (handshake)	No

	TCP	UDP
Reliability / ordering	Yes	No (application handles it)
Header	20+ bytes	8 bytes
Speed / overhead	More overhead	Minimal
Examples	HTTP/1.1, HTTP/2, SSH, databases	DNS, NTP, VoIP, QUIC

ICMP

Control messages for IP: **echo request/reply** (`ping`), **destination unreachable**, **time exceeded** (used by `traceroute`), "fragmentation needed" (Path MTU discovery). Blocking all ICMP can break PMTU discovery and makes troubleshooting harder; many clouds block ping by default in security groups.

MTU

Maximum packet size on a link (Ethernet 1500 bytes; AWS within a VPC supports 9001 "jumbo frames"; VPNs/overlays reduce it). MTU mismatches cause "small requests work, large ones hang". Test: `ping -M do -s 1472 host` (Linux).

2.9 DNS in depth

DNS (Domain Name System) is the internet's distributed phone book: it maps names to IPs (and more).

Name hierarchy

```
                    . (root)
          ┌─────────┬┴┬──────────┐
        com         org         in      ← TLDs (top-level domains)
         │
      example                           ← domain (registered by you)
     ┌───┴────┐
    www      api                         ← subdomains / hostnames
FQDN: api.example.com.
```

Resolution step by step

```
App → OS stub resolver (checks /etc/hosts, local cache)
    → Recursive resolver (ISP, 8.8.8.8, 1.1.1.1, VPC resolver 10.0.0.2)
        ├─ asks a ROOT server:          "who handles .com?"         → .com TLD servers
        ├─ asks the .com TLD server:    "who handles example.com?"  → NS ns1.example-dns.net
        └─ asks the AUTHORITATIVE server: "A record for api.example.com?" → 203.0.113.10 (TTL 300)
    ← answer cached by the resolver and the client for TTL seconds
```

Server role	Job
Recursive resolver	Does the lookups for clients and caches results
Root servers	Know the TLD servers
TLD servers	Know the authoritative servers for each domain
Authoritative server	Holds the actual records for a zone (e.g. Route 53, Cloudflare, your DNS provider)

Record types

Record	Purpose	Example
A	Name → IPv4	`api.example.com. 300 IN A 203.0.113.10`
AAAA	Name → IPv6	`api AAAA 2001:db8::10`
CNAME	Alias to another name (not allowed at the zone apex)	`www CNAME example.com.`
ALIAS / ANAME	Provider-specific apex alias (Route 53 Alias)	`example.com → ALB`
MX	Mail servers with priority	`MX 10 mail.example.com.`
TXT	Text: SPF, DKIM, DMARC, domain verification	`"v=spf1 include:_spf.google.com ~all"`
NS	Authoritative name servers of a zone	`NS ns-123.awsdns-15.com.`
SOA	Zone metadata (primary NS, serial, timers)	Created automatically
PTR	Reverse lookup IP → name	`10.113.0.203.in-addr.arpa PTR api.example.com.`
SRV	Service location with port	`_sip._tcp SRV 10 60 5060 sip.example.com.`
CAA	Which CAs may issue certificates	`CAA 0 issue "letsencrypt.org"`

TTL and caching

TTL (seconds) tells resolvers how long to cache an answer. High TTL = fewer queries, slow changes; low TTL = fast changes, more queries. **Before a migration, lower the TTL** (e.g. to 60) a day in advance, switch, then raise it again. **Negative caching:** "does not exist" (NXDOMAIN) answers are cached too.

DNS in different environments

Environment	Detail
Linux	`/etc/hosts`, `/etc/resolv.conf` (often `127.0.0.53` = systemd-resolved), `resolvectl status`
Windows	`ipconfig /displaydns`, `ipconfig /flushdns`, `Resolve-DnsName`
AWS VPC	Route 53 Resolver at VPC base + 2; private hosted zones; Resolver endpoints for hybrid DNS
Docker	Embedded DNS (`127.0.0.11`) resolves container names on user-defined networks
Kubernetes	**CoreDNS**: `service.namespace.svc.cluster.local`; pods' `/etc/resolv.conf` has search domains and `ndots:5`
Split-horizon DNS	Same name resolves differently inside vs outside (internal IP inside the VPC, public IP outside)

Security: DNSSEC signs records (authenticity); DNS over HTTPS/TLS encrypts queries; SPF/DKIM/DMARC TXT records protect email from spoofing.

DNS commands

```
dig api.example.com                 # status, ANSWER section, TTL, which server answered
dig +short api.example.com          # just the IPs
dig @8.8.8.8 api.example.com        # ask a specific resolver
dig example.com NS / MX / TXT       # other record types
dig +trace api.example.com          # follow the delegation from the root
dig -x 203.0.113.10                 # reverse lookup
nslookup api.example.com            # simple lookup (Windows too)
getent hosts api.example.com        # resolve like applications do (includes /etc/hosts)
```

Response codes: `NOERROR` (exists; check the ANSWER), `NXDOMAIN` (name doesn't exist), `SERVFAIL` (resolver couldn't get an answer: DNSSEC failure, broken authoritative servers), `REFUSED` (server won't answer you).

2.10 DHCP

Gives devices their network configuration automatically: IP address, subnet mask, default gateway, DNS servers, lease time.

DORA: Discover (client broadcast: "any DHCP server?") → **O**ffer (server proposes an IP) → **R**equest (client accepts) → **A**cknowledge (server confirms). Leases are renewed before they expire. Routers forward DHCP to another subnet with a **DHCP relay** (`ip helper-address`).

A host with `169.254.x.x` means DHCP failed. In AWS, DHCP options sets define the DNS servers and domain name for a VPC.

2.11 HTTP in depth

Request and response structure

```
REQUEST                                 RESPONSE
GET /api/products?page=2 HTTP/1.1       HTTP/1.1 200 OK
Host: shop.example.com                  Content-Type: application/json
User-Agent: curl/8.5.0                  Content-Length: 512
Accept: application/json                Cache-Control: max-age=60
Authorization: Bearer eyJhbGciOi...     Set-Cookie: session=abc123; HttpOnly; Secure

(optional body, e.g. JSON for POST)     {"items": [...], "page": 2}
```

A request = **method + path + version**, **headers**, optional **body**. A response = **status code**, headers, body.

Methods

Method	Purpose	Safe (no change)	Idempotent (repeat = same result)
GET	Read	Yes	Yes
HEAD	Like GET, headers only	Yes	Yes
POST	Create / trigger action	No	**No**
PUT	Replace a resource	No	Yes
PATCH	Partial update	No	Not necessarily
DELETE	Delete	No	Yes
OPTIONS	Allowed methods / CORS preflight	Yes	Yes

Idempotency matters for **retries**: retrying a GET or PUT is safe; retrying a POST may create duplicates (use idempotency keys).

Status codes

Code	Meaning	Typical cause
200 OK / 201 Created / 204 No Content	Success	
301 / 308	Permanent redirect	HTTP → HTTPS, moved URL
302 / 307	Temporary redirect	Login redirects
304 Not Modified	Use your cached copy	Conditional request with ETag
400 Bad Request	Malformed request	Invalid JSON, missing parameter
401 Unauthorized	Not authenticated	Missing/expired token
403 Forbidden	Authenticated but not allowed	Permissions, WAF block, S3 policy
404 Not Found	No such path/resource	Wrong URL, wrong Ingress path
405 Method Not Allowed	Wrong method	POST to a GET-only endpoint
408 / 429	Request timeout / Too many requests	Slow client / rate limiting
500 Internal Server Error	Unhandled error in the app	Bug, exception: check app logs
502 Bad Gateway	Proxy/LB got an invalid response from the backend	Backend crashed, wrong port
503 Service Unavailable	Backend unavailable/overloaded	No healthy targets, maintenance
504 Gateway Timeout	Backend didn't respond in time	Slow app/database, timeouts

Rule of thumb: 4xx = the client (request) is wrong; 5xx = the server side is wrong. 502/503/504 point at the **path between proxy/LB and backend**.

Important headers

Header	Purpose
`Host`	Which site (one IP can host many sites: virtual hosting, Ingress host rules)
`Content-Type` / `Accept`	Body format sent / wanted (`application/json` , `text/html`)
`Authorization`	Credentials (`Bearer <token>` , `Basic <base64>`)
`Cookie` / `Set-Cookie`	Session state in the browser (`HttpOnly` , `Secure` , `SameSite` flags)
`Cache-Control` , `ETag` , `Last-Modified` , `Expires`	Caching by browsers and CDNs
`X-Forwarded-For` , `X-Forwarded-Proto` , `Forwarded`	Original client IP and scheme when behind proxies/load balancers

Header	Purpose
`Location`	Target of a redirect
`Strict-Transport-Security` (HSTS)	Force HTTPS in browsers
`Access-Control-Allow-Origin`	CORS: which other websites may call this API from a browser
`User-Agent`	Client software

Statelessness, sessions and tokens

HTTP is **stateless**: each request stands alone. State is added with **cookies + server-side sessions** (store sessions in Redis/DB, not in one server's memory, so any server can handle the next request) or **tokens** (e.g. JWT carried in the `Authorization` header).

REST APIs and JSON

REST style: resources as URLs (`/orders/42`), methods as actions, JSON bodies, status codes for results, versioning (`/v1/`). Other API styles: **GraphQL** (one endpoint, client chooses fields), **gRPC** (binary Protobuf over HTTP/2, fast service-to-service), **WebSockets** (two-way persistent connection for real-time apps).

HTTP versions

Version	Key change
HTTP/1.1	Text protocol, keep-alive connections, one request at a time per connection
HTTP/2	Binary, **multiplexing** many requests over one TCP connection, header compression
HTTP/3	Runs over **QUIC (UDP)**: faster setup, no head-of-line blocking, better on mobile networks

curl for HTTP debugging

```
curl -v https://shop.example.com/health               # full request/response incl. TLS
curl -I https://shop.example.com                      # headers only
curl -L http://shop.example.com                       # follow redirects
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://shop.example.com
curl -H "Host: shop.example.com" http://10.0.1.25/    # test a virtual host directly on an IP
curl --resolve shop.example.com:443:10.0.1.25 https://shop.example.com/  # keeps SNI right
curl -X POST -H "Content-Type: application/json" -d '{"qty":2}' https://api.example.com/orders
# where is the time spent?
curl -o /dev/null -s https://shop.example.com -w \
  "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} total:%{time_total}\n"
```

2.12 TLS and HTTPS in depth

TLS (Transport Layer Security) provides **encryption** (no eavesdropping), **integrity** (no tampering) and **authentication** (you're talking to the real server). **HTTPS = HTTP over TLS** (port 443). SSL is the obsolete predecessor; use **TLS 1.2 or 1.3** only.

TLS 1.3 handshake (simplified)

```
Client                                              Server
  | ClientHello: TLS versions, cipher suites, SNI (hostname), key share →|
  |← ServerHello: chosen cipher, key share; Certificate; signature; Finished
  |  client verifies the certificate chain and hostname, derives session keys
  | Finished ──────────────────────────────────────────────────────────→|
  | ═══════════ encrypted application data (symmetric keys, e.g. AES-GCM) ═══════════
```

Asymmetric cryptography (certificates, key exchange) is used only to agree on **symmetric session keys**; bulk data uses fast symmetric encryption.

Certificates and the chain of trust

```
Root CA certificate (pre-installed in OS/browser trust store)
   └── signs Intermediate CA certificate
          └── signs Server (leaf) certificate: CN/SAN = shop.example.com, valid from/to, public key
```

Term	Meaning
CA (Certificate Authority)	Trusted organization that signs certificates (Let's Encrypt, DigiCert, AWS ACM's CA)
SAN (Subject Alternative Name)	The hostnames the certificate is valid for (`shop.example.com` , `*.example.com`)
Wildcard	`*.example.com` covers one level of subdomains
Chain / bundle	Server must send its certificate **plus intermediates**
Private key	Kept secret on the server/load balancer; never shared or committed
CSR	Certificate Signing Request sent to a CA
SNI	Hostname sent in ClientHello so one IP can serve many certificates
Validity	Public certificates are short-lived (Let's Encrypt 90 days): **automate renewal** (certbot, ACM, cert-manager)
mTLS (mutual TLS)	**Both** sides present certificates: service-to-service auth, service meshes, partner APIs
Self-signed	Not trusted by default: fine for labs, not for public services

TLS commands

```
# handshake + certificate chain
openssl s_client -connect shop.example.com:443 -servername shop.example.com </dev/null
echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName   # cert details
openssl x509 -in cert.pem -noout -text                       # inspect a certificate file
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 30 \
  -subj "/CN=lab.local"                                       # self-signed cert for labs
```

Common TLS errors

Error	Cause
Certificate has expired	Renewal failed: automate and monitor expiry
Hostname mismatch	Name not in the certificate SAN; wrong cert served (SNI)
Unable to get local issuer / incomplete chain	Server not sending intermediate certificates
Self-signed certificate	Untrusted CA (labs, internal CAs not installed in the trust store)
Handshake failure / protocol version	Client and server share no TLS version/cipher (old clients)
Certificate not yet valid	Clock wrong on the client/server (check NTP)

TLS termination: decrypting at the load balancer/ingress (simpler certificate management) vs **end-to-end** (re-encrypt to backends) for stricter security.

2.13 SSH (protocol view)

SSH (TCP 22) gives encrypted remote shells, file transfer (SCP/SFTP) and tunnels. Server proves identity with a **host key** (stored in `known_hosts`); users authenticate with **key pairs** (preferred) or passwords. Full practical coverage is in the Linux notes (keys, config, jump hosts, port forwarding, hardening).

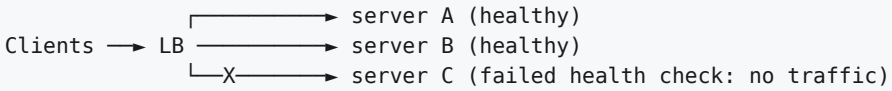
2.14 Firewalls and network security

Type	How it works	Examples
Packet filter (stateless)	Checks each packet against rules (IP, port, protocol); return traffic needs explicit rules	AWS NACLs, simple ACLs
Stateful firewall	Tracks connections; replies to allowed connections pass automatically	AWS Security Groups, iptables/nftables conntrack, ufw, firewalld
NGFW (next-generation)	Adds application awareness, user identity, IPS, TLS inspection	Palo Alto, Fortinet, AWS Network Firewall
WAF	Inspects HTTP requests for attacks (SQL injection, XSS, bots, rate limits)	AWS WAF, Cloudflare, ModSecurity
Host firewall	On the server itself	ufw, firewalld, Windows Firewall

Design principles: default deny inbound, allow only what's needed (least privilege), segment networks (public / app / data tiers), restrict admin ports (SSH/RDP) to VPN/bastion or remove them (SSM), log denied traffic, defense in depth (cloud SG + host firewall + app auth).

Zones: internet (untrusted) → **DMZ** (public-facing load balancers/proxies) → internal app network → data network.

2.15 Load balancing



Type	Works on	Can route by	Examples
Layer 4	IP + TCP/UDP port	Connection	AWS NLB, HAProxy (TCP mode), kube-proxy
Layer 7	HTTP(S) content	Host, path, headers, cookies, methods	AWS ALB, nginx, HAProxy (HTTP), Envoy, Kubernetes Ingress controllers
DNS / global	DNS answers	Geography, latency, health	Route 53, Cloudflare, Global Accelerator

Algorithm	Behaviour
Round robin	Each server in turn
Weighted round robin	Bigger servers get more
Least connections / least outstanding requests	Server with the fewest active requests
IP hash / consistent hashing	Same client → same server (affinity, caches)
Random with two choices	Pick two at random, choose the less loaded

Feature	Detail
Health checks	Active probes (`GET /health` every N seconds, healthy/unhealthy thresholds); passive (observe failures)
Sticky sessions	Keep a user on one server (cookie); avoid when possible: store sessions centrally
TLS termination / offload	LB handles certificates and decryption
Connection draining	Let in-flight requests finish before removing a server (deployments)
Timeouts	Idle/backend timeouts must be consistent with the app (cause of 502/504)
Client IP	Lost behind L7 proxies → use `X-Forwarded-For` or PROXY protocol

2.16 Proxies, reverse proxies, API gateways and CDNs

Component	Sits in front of	Purpose	Examples
Forward proxy	**Clients**	Outbound filtering, caching, hiding client IPs; corporate internet access (`HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` env variables)	Squid, Zscaler
Reverse proxy	**Servers**	Single entry point, TLS termination, load balancing, caching, compression, routing by path/host	nginx, HAProxy, Envoy, Traefik, ALB
API gateway	APIs	Reverse proxy + authentication, rate limiting, API keys, request transformation, analytics	AWS API Gateway, Kong, Apigee
Service mesh	Service-to-service traffic	mTLS, retries, timeouts, traffic splitting, telemetry via sidecar/ambient proxies	Istio, Linkerd
CDN	Websites/APIs	Caches content at edge locations near users; DDoS absorption	CloudFront, Cloudflare, Akamai

Caching basics: cache **hit** (served from cache) vs **miss** (fetched from origin); controlled by `Cache-Control` headers and TTLs; **invalidation** clears cached objects; use **versioned file names** (`app.3f9a1c.js`) for static assets so you rarely need invalidation.

2.17 VPNs and private connectivity

Type	Use	Technology
Site-to-site VPN	Connect an office/data center to a cloud VPC or another office	IPsec tunnels (IKE for keys, ESP for encryption), often with BGP
Remote-access VPN	Employees reach internal networks	OpenVPN, WireGuard, IPsec clients, TLS VPNs
Zero-trust access (ZTNA)	Per-application access based on identity and device, not network location	Cloudflare Access, Zscaler, AWS Verified Access
Dedicated links	Private, consistent bandwidth to cloud	AWS Direct Connect, Azure ExpressRoute
Overlay networks	Virtual networks over existing IP networks	VXLAN, WireGuard meshes (Tailscale)

2.18 Networking in cloud, containers and Kubernetes

Cloud (AWS mapping)

Traditional	AWS
Data center network	VPC
Subnet / VLAN	Subnet (one AZ)
Router + routing table	Route tables
Internet edge	Internet Gateway
NAT router	NAT Gateway
Host firewall / stateful firewall	Security Group
Subnet ACL	Network ACL
Load balancer	ALB / NLB
DNS server	Route 53 / VPC Resolver
VPN / leased line	Site-to-Site VPN / Direct Connect

Details: AWS notes, VPC chapter.

Docker networking

```
Host eth0 10.0.1.25 ── docker0 bridge 172.17.0.1 ─┬─── container A 172.17.0.2
                                                  └── container B 172.17.0.3

docker run -p 8080:80 nginx   →  host:8080 ──(DNAT iptables)──► container:80
Containers → internet          →  source NAT (masquerade) to the host's IP
```

- **Bridge** (default): private network per host; publish ports with `-p host:container`.
- **User-defined bridge:** containers resolve each other **by name** (embedded DNS).
- **Host network:** container shares the host's network stack (no isolation, no `-p`).
- Inside a container, `localhost` is the **container itself**, not the host.

Kubernetes networking model

Rule / object	Meaning
Every **pod** gets its own IP	Pods talk to each other directly across nodes without NAT (implemented by a **CNI** plugin: AWS VPC CNI, Calico, Cilium, Flannel)
Service (ClusterIP)	Stable virtual IP + DNS name in front of changing pods; kube-proxy (iptables/IPVS) or eBPF load-balances to pod IPs
NodePort / LoadBalancer	Expose a Service on every node's port (30000–32767) / through a cloud load balancer
Ingress / Gateway API	Layer 7 HTTP(S) routing by host/path through an ingress controller
CoreDNS	`my-svc.my-namespace.svc.cluster.local` names
NetworkPolicy	Firewall rules between pods (needs a CNI that supports it)

Details: Docker and Kubernetes notes.

2.19 Networking tools reference

Tool	Purpose	Example
`ip a`, `ip r`, `ip neigh`	Addresses, routes, ARP (Linux)	`ip route get 8.8.8.8`
`ipconfig /all`, `route print`, `arp -a`	Same on Windows	
`ping`	Reachability + latency (ICMP)	`ping -c 4 10.0.11.5`
`traceroute` / `tracert` / `mtr`	Path, hop where it fails or slows	`mtr -rwc 50 api.example.com`
`dig` / `nslookup` / `Resolve-DnsName`	DNS	`dig +short api.example.com`
`curl` / `wget`	HTTP(S) testing	`curl -v https://…`
`nc` / `Test-NetConnection`	TCP port test	`nc -zv db 5432`
`ss` / `netstat`	Listening ports, connections	`ss -tulpn`, `ss -tan state established`
`tcpdump`	Packet capture on the CLI	`sudo tcpdump -i any -nn host 10.0.1.5 and port 443`
Wireshark	GUI packet analysis (filters: `dns`, `tcp.port == 443`, `http`, `tcp.flags.reset == 1`)	Open a `.pcap`
`openssl s_client`	TLS/certificate checks	see 2.12
`iperf3`	Bandwidth test between two hosts	`iperf3 -s` / `iperf3 -c server`
`ethtool`	NIC speed/duplex/errors	`ethtool eth0`

2.20 Network troubleshooting

Layered method

```
1. Name    → does DNS resolve correctly?          dig / nslookup / getent hosts
2. Route   → can I reach the IP?                  ping, traceroute/mtr, ip route
3. Port    → is the port open and allowed?        nc -zv / Test-NetConnection, ss -tulpn on server
4. TLS     → is the certificate valid?            openssl s_client, curl -v
5. App     → does the application answer right?   curl -v, app logs, status codes
At each step: firewalls / security groups / NACLs / proxies / NAT on the path?
```

Scenarios

Symptom	Likely causes and checks
Website down for everyone	DNS record changed/expired domain? LB health (no healthy targets → 503)? Certificate expired? App crashed (502)? Recent deploy?
Works by IP, not by name	DNS: wrong record, not propagated (TTL), wrong resolver, `/etc/hosts` override
`Connection timed out`	Security group/NACL/firewall dropping, wrong IP, no route, host down
`Connection refused`	Service not running or listening on `127.0.0.1`/different port; host firewall REJECT

Symptom	Likely causes and checks
Works inside the server, not from outside	Bound to `127.0.0.1`; host firewall; security group; LB/target port mismatch
Intermittent failures	One bad backend behind a LB, DNS round-robin to a dead IP, packet loss, connection pool exhaustion, NAT port exhaustion
Slow responses	Check timing (`curl -w`): slow DNS? slow TCP connect (network/latency)? slow TLS? slow server (app/DB)?
Large requests hang, small work	MTU problem across VPN/overlay
Private server can't reach the internet	No NAT/route, no DNS, outbound rules, proxy settings
Container can't reach another container	Not on the same user-defined network, using `localhost`, wrong port
Pod can't reach a Service	Service selector doesn't match pod labels (no endpoints), wrong port/targetPort, NetworkPolicy, CoreDNS issues

2.21 Interview questions and answers

#	Question	Answer
1	Explain the OSI model and where TCP, IP, HTTP sit	7 layers; HTTP = 7, TCP = 4, IP = 3, Ethernet = 2
2	What happens when you type a URL in a browser?	DNS resolution → TCP handshake → TLS handshake → HTTP request → server/app/DB → response → render; caches at several steps
3	TCP vs UDP?	Reliable ordered connection with handshake vs connectionless best-effort; HTTP/SSH vs DNS/VoIP/QUIC
4	Explain the TCP three-way handshake	SYN → SYN-ACK → ACK
5	Connection refused vs timed out?	Refused = reached the host but no listener/REJECT; timeout = no answer (dropped by firewall, routing, host down)
6	What is a subnet? How many hosts in /26?	A range of addresses sharing a network prefix; 64 addresses, 62 usable (AWS: 59)
7	Private vs public IP?	Private RFC 1918 ranges are not internet-routable and need NAT
8	What is NAT and where is it used?	Address translation: home routers, NAT Gateway, Docker port publishing, LBs
9	What is a default gateway?	The router a host sends traffic to for destinations outside its subnet
10	Explain DNS resolution	Stub → recursive resolver → root → TLD → authoritative; cached per TTL
11	A vs CNAME vs Alias?	A = name → IP; CNAME = name → name (not at apex); Alias = provider feature for apex → AWS resource
12	Why lower TTL before a migration?	So resolvers cache the old answer briefly and the switch propagates quickly
13	What does DHCP do?	Assigns IP, mask, gateway, DNS automatically (DORA)
14	HTTP 401 vs 403?	Not authenticated vs authenticated but not permitted
15	HTTP 502 vs 503 vs 504?	Bad response from backend / no available backend / backend timed out

#	Question	Answer
16	What is idempotency and why does it matter?	Repeating a request has the same effect; makes retries safe (GET/PUT/DELETE yes, POST no)
17	How does HTTPS work?	TLS handshake authenticates the server with a CA-signed certificate and agrees on symmetric keys; data is then encrypted
18	What is a certificate chain?	Leaf cert signed by an intermediate signed by a trusted root; servers must send intermediates
19	What is SNI?	Hostname in the TLS ClientHello so one IP can serve multiple certificates
20	What is mTLS?	Both client and server authenticate with certificates
21	L4 vs L7 load balancer?	Transport (IP/port) vs application (HTTP host/path/headers) routing
22	Forward vs reverse proxy?	In front of clients (outbound) vs in front of servers (inbound)
23	What does a CDN do?	Caches content at edge locations near users: faster, less origin load, DDoS absorption
24	Stateful vs stateless firewall?	Tracks connections and allows replies vs evaluates each packet independently
25	How do you find what's listening on a port?	`ss -tulpn` / `lsof -i :port` (Linux), `netstat -ano` (Windows)
26	How do you troubleshoot "cannot reach service X"?	DNS → ping/route → port test → firewall/SG → service listening address → TLS → app logs
27	What is MTU and what breaks when it's wrong?	Max packet size; large transfers hang while small ones work
28	How do Docker containers talk to each other?	Same user-defined network, by container name via embedded DNS
29	How does a Kubernetes Service work?	Stable virtual IP/DNS name; kube-proxy/eBPF forwards to the pods matching its selector
30	What is `0.0.0.0` vs `127.0.0.1` for a server?	Listen on all interfaces vs only local loopback

2.22 Self-check: I can…

- [ ] Explain the OSI/TCP-IP layers and map common problems to layers.
- [ ] Explain MAC, ARP, IPv4 classes of addresses, private vs public and special ranges.
- [ ] Convert binary/decimal/hex and subnet any address in under a minute; design VLSM and VPC plans.
- [ ] Explain IPv6 basics, routing, default gateways, longest-prefix match, NAT types.
- [ ] Describe TCP handshake, states, errors (refused/timeout/reset) and UDP use cases.
- [ ] Explain DNS resolution, record types, TTL, and debug with `dig`.
- [ ] Read HTTP requests/responses; explain methods, status codes, headers, caching, REST, HTTP/2/3.
- [ ] Explain TLS handshake, certificates, chain of trust, SNI, mTLS; inspect certs with `openssl`.
- [ ] Compare stateful/stateless firewalls, WAFs and network segmentation.
- [ ] Explain L4/L7 load balancing, algorithms, health checks, draining and timeouts.
- [ ] Explain proxies, reverse proxies, API gateways, service meshes and CDNs.
- [ ] Describe VPNs, Direct Connect-style links and zero-trust access.
- [ ] Explain Docker and Kubernetes networking basics.

- [ ] Troubleshoot connectivity step by step with the right tool at each layer.

REMEMBER

- OSI layers 7 → 1: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.
- Status codes: **2xx** OK · **3xx** moved · **4xx** you made a mistake · **5xx** the server made a mistake.
- Troubleshoot: link → IP → gateway → internet (8.8.8.8) → DNS → port → application.
- Subnet hosts = $2^{\text{host bits}} - 2$. Block size = 256 − mask octet.